CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

# Validation Report


**Team 40**

| | | |
|---|---|---|
| **Sajjad Nakhwa** | 23B0702 | Project Lead and Finance Modeller |
| **Dependar Arya** | 23B0331 | Data Lead |
| **Neel Shah** | 23B4244 | AI and Tool Developer |
| **Akshat Karkar** | 23B4229 | Validation Engineer |
| **Pearsh Gumasta** | 23B4232 | Documentation and Reflection Lead |



Semester course project · November 2026

Benchmark case: reporting periods 5 and 6 of the case study project

---

# 1. Summary

| | |
|---|---|
| Measures compared | 25 per period, 2 periods, **50 comparisons** |
| Agreement | **50 of 50**, within tolerance |
| Tolerance | ₹0.01 on currency, $1 \times 10^{-9}$ on dimensionless indices |
| Discrepancies found during development | 6, all resolved — section 5 |
| Automated tests | **151**, all passing |

The headline result is that two independently written implementations of the EVM definitions — one in Python, one in typed Excel formulas — agree on every measure, at both reporting periods, to nine decimal places.

**What that does and does not establish** is set out in section 6, and matters more than the result itself.

---

# 2. Validation strategy

## 2.1 The approach, and the trap it avoids

The obvious way to validate a calculation engine is to write a second implementation in the same language and compare. We did not do that, and the reason reshaped the whole approach.

A Python script that re-implements the intended Excel logic and compares it with the engine is **checking our understanding against itself**. If we misread a formula — used EV/PV where the definition says EV/AC, or applied TCPI against the wrong denominator — the re-implementation embeds the same misreading, agrees perfectly with the engine, and the validation passes while both are wrong. The test would be measuring our consistency, not our correctness.

This argument is taken from the verification notes in `Tahir-yamin/excel-scurve-builder`, which states it as: *a self-check that re-implements the intended logic in Python only validates your own understanding — it will agree with your bug.*

So the benchmark is built in a **different execution environment with different semantics**, and it is **executed rather than re-implemented**:

```
Raw inputs  (budgets, % complete, actual cost, time-phased budget)
     |
     +---------------------------+
     |                           |
Python engine              Excel formulas
(evm package)       (EVM_Manual_Benchmark.xlsx)
     |                           |
     |                    evaluated by
     |                  LibreOffice headless
     |                           |
     +-----------> compare <-----+
                 50 measures
```

`scripts/validate_benchmark.py` drives LibreOffice over the workbook, which loads it, recalculates every cell with a real spreadsheet engine, and writes the computed values back. We then read those values. At no point does Python compute what the Excel formula "should" give.

### 2.2 What is in the benchmark workbook

`benchmark/EVM_Manual_Benchmark.xlsx` contains, for periods 5 and 6:

- **Inputs sheet** — the only typed numbers in the workbook: 19 work package budgets, actual cost for six periods, percent complete for six periods.
- **Baseline sheet** — the time-phased budget, with a reconciliation column asserting each package's phasing sums to its budget, and a cumulative planned value row that Earned Schedule interpolates along.
- **Calc P5** / **Calc P6** — per work package: `EV = budget × % complete`, cumulative PV, cumulative AC, CV, SV, CPI, SPI.
- **Indices sheet** — all 25 measures, as formulas, both periods side by side.

Every green cell is a formula. Open the file, click any of them, and the arithmetic is visible.

**On the workbook being script-placed.** The formulas are laid out by `benchmark/build_benchmark.py` rather than typed cell by cell. 19 packages across 12 baseline periods is 228 planned-value cells, and hand-typing them would introduce exactly the transcription errors the benchmark exists to catch. The script places formulas; Excel evaluates them. **The formulas are the manual method** — no value is carried across from Python.

---

## 3. The hand calculation

Both periods were worked by hand from the raw inputs before the engine was run. Reproduced here so the arithmetic can be followed without opening anything.

### 3.1 Earned value — Period 6

`EV = cumulative percent complete × budget at completion`

| WBS | Work package | Budget (₹ lakh) | % complete | EV (₹ lakh) |
|---|---|---|---|---|
| 1.1 | Site mobilisation and enabling works | 25.00 | 100% | 25.00 |
| 1.2 | Site office, stores and labour accommodation | 18.00 | 100% | 18.00 |

| WBS | Work package | Budget (₹ lakh) | % complete | EV (₹ lakh) |
|---|---|---|---|---|
| 2.1 | Excavation and earthwork | 42.00 | 100% | 42.00 |
| 2.2 | PCC and footings | 68.00 | 100% | 68.00 |
| 2.3 | Foundation RCC up to plinth | 95.00 | 100% | 95.00 |
| 2.4 | Plinth beams and backfilling | 38.00 | 100% | 38.00 |
| 3.1 | RCC columns, ground to fourth floor | 145.00 | 22% | 31.90 |
| 3.2 | RCC beams and slabs, ground to fourth | 210.00 | 14% | 29.40 |
| 3.3 | Staircase and lift core RCC | 55.00 | 0% | 0.00 |
| — | All packages downstream of the frame | 609.00 | 0% | 0.00 |
| | **Total EV** | **1,400.00** | | **347.30** |

**EV = ₹3,47,30,000**

## 3.2 Actual cost — Period 6

Cumulative sum of cost booked per period, ₹ lakh:

| WBS | P1 | P2 | P3 | P4 | P5 | P6 | Total | Budget | Variance |
|---|---|---|---|---|---|---|---|---|---|
| 1.1 | 15.0 | 12.0 | — | — | — | — | 27.0 | 25.0 | −2.0 |
| 1.2 | 9.5 | 9.0 | — | — | — | — | 18.5 | 18.0 | −0.5 |
| 2.1 | 11.0 | 15.0 | 16.5 | 11.0 | 7.5 | — | **61.0** | 42.0 | **−19.0** |
| 2.2 | — | 11.0 | 19.5 | 25.0 | 18.5 | — | 74.0 | 68.0 | −6.0 |
| 2.3 | — | — | 19.0 | 30.0 | 34.0 | 25.0 | **108.0** | 95.0 | **−13.0** |
| 2.4 | — | — | — | 8.5 | 16.0 | 16.5 | 41.0 | 38.0 | −3.0 |
| 3.1 | — | — | — | — | 13.0 | 23.0 | 36.0 | — | — |
| 3.2 | — | — | — | — | 12.0 | 21.0 | 33.0 | — | — |
| | | | | | | | **398.5** | | |

**AC = ₹3,98,50,000**

## 3.3 Planned value — Period 6

Packages whose baseline finish is Period 5 or earlier are fully planned:

```
1.1 25 + 1.2 18 + 2.1 42 + 2.2 68 + 2.3 95 + 2.4 38  =  286.000 lakh
```

Packages still in progress against the baseline, phased by smoothstep $3t^2 - 2t^3$ where S-shaped:

```
3.1  P5–P9,  n = 5, 2 elapsed, t = 0.4
     3(0.16) − 2(0.064) = 0.352          145.00 × 0.352   =   51.040
3.2  P5–P10, n = 6, 2 elapsed, t = 1/3
     3(1/9) − 2(1/27)  = 7/27            210.00 × 7/27    =   54.444
3.3  P6–P10, n = 5, linear, 1 elapsed    55.00 × 0.20     =   11.000
                                                            ---------
                                                Total       402.484 lakh
```

**PV = ₹4,02,48,444**

### 3.4 Variances, indices and forecasts — Period 6

```
CV   = EV − AC = 3,47,30,000 − 3,98,50,000        = −51,20,000
SV   = EV − PV = 3,47,30,000 − 4,02,48,444        = −55,18,444
CPI  = EV / AC = 3,47,30,000 / 3,98,50,000        =  0.871518
SPI  = EV / PV = 3,47,30,000 / 4,02,48,444        =  0.862890
```

Earned Schedule. Cumulative PV reaches ₹3,16,63,556 at Period 5 and ₹4,02,48,444 at Period 6; current EV of ₹3,47,30,000 falls between them:

```
ES     = 5 + (3,47,30,000 − 3,16,63,556) / (4,02,48,444 − 3,16,63,556)
       = 5 + 30,66,444 / 85,84,889
       = 5.357191 months
SV(t)  = 5.357191 − 6 = −0.642809 months   ≈ 19 days behind
SPI(t) = 5.357191 / 6 =  0.892865
```

Forecasts, with remaining work BAC − EV = ₹10,52,70,000:

```
EAC1 = BAC / CPI                      = 14,00,00,000 / 0.871518    = 16,06,39,217
EAC2 = AC + (BAC − EV)                = 3,98,50,000 + 10,52,70,000 = 14,51,20,000
EAC3 = AC + (BAC − EV)/(CPI × SPI)    ÷ 0.7520248                  = 17,98,32,093
EAC4 = AC + (BAC − EV)/(0.8CPI+0.2SPI) ÷ 0.8697857                 = 16,08,78,845

TCPI(BAC) = (BAC − EV)/(BAC − AC) = 10,52,70,000 / 10,01,50,000    = 1.051123
```

### 3.5 Period 5, in brief

Worked the same way:

```
EV = 25 + 18 + 42 + 68 + (0.80×95) + (0.62×38) + (0.08×145) + (0.05×210)
   = 25 + 18 + 42 + 68 + 76.00 + 23.56 + 11.60 + 10.50  =  274.66 lakh
AC = 27 + 18.5 + 61 + 74 + 83 + 24.5 + 13 + 12          =  313.00 lakh
PV                                                       =  316.636 lakh

CPI = 274.66 / 313.00    = 0.877508
SPI = 274.66 / 316.636   = 0.867433
ES  = 4 + (274.66 − 242.370) / (316.636 − 242.370) = 4.434788
SPI(t) = 4.434788 / 5    = 0.886958
```

---

## 4. Comparison result

All 50 comparisons agree. The complete machine-generated table is at `generated/benchmark-comparison.md`; the headline measures:

### Period 6

| Measure | Tool (Python) | Manual benchmark (Excel) | Difference |
|---|---|---|---|
| Planned value (PV) | 40,248,444.43 | 40,248,444.43 | 0.00 |
| Earned value (EV) | 34,730,000.00 | 34,730,000.00 | 0.00 |
| Actual cost (AC) | 39,850,000.00 | 39,850,000.00 | 0.00 |

| Measure | Tool (Python) | Manual benchmark (Excel) | Difference |
|---|---|---|---|
| Cost variance (CV) | −5,120,000.00 | −5,120,000.00 | 0.00 |
| Schedule variance (SV) | −5,518,444.43 | −5,518,444.43 | 0.00 |
| CPI | 0.871518193 | 0.871518193 | 0.000000000 |
| SPI | 0.862890492 | 0.862890492 | 0.000000000 |
| Earned schedule (ES) | 5.357190932 | 5.357190932 | 0.000000000 |
| SPI(t) | 0.892865155 | 0.892865155 | 0.000000000 |
| SV(t) | −0.642809068 | −0.642809068 | 0.000000000 |
| EAC method 1 | 160,639,216.82 | 160,639,216.82 | 0.00 |
| EAC method 2 | 145,120,000.00 | 145,120,000.00 | 0.00 |
| EAC method 3 | 179,832,092.74 | 179,832,092.74 | 0.00 |
| EAC method 4 | 160,878,844.80 | 160,878,844.80 | 0.00 |
| TCPI to BAC | 1.051123315 | 1.051123315 | 0.000000000 |

### Period 5

| Measure | Tool (Python) | Manual benchmark (Excel) | Difference |
|---|---|---|---|
| Planned value (PV) | 31,663,555.54 | 31,663,555.54 | 0.00 |
| Earned value (EV) | 27,466,000.00 | 27,466,000.00 | 0.00 |
| Actual cost (AC) | 31,300,000.00 | 31,300,000.00 | 0.00 |
| CPI | 0.877507987 | 0.877507987 | 0.000000000 |
| SPI | 0.867432590 | 0.867432590 | 0.000000000 |
| Earned schedule (ES) | 4.434788249 | 4.434788249 | 0.000000000 |
| SPI(t) | 0.886957650 | 0.886957650 | 0.000000000 |
| EAC method 1 | 159,542,707.35 | 159,542,707.35 | 0.00 |
| EAC method 3 | 179,141,698.34 | 179,141,698.34 | 0.00 |
| TCPI to BAC | 1.035271389 | 1.035271389 | 0.000000000 |

## 4.1 Algebraic identities, checked on both sides

Two identities hold only if the entire calculation chain is intact. They are asserted against the Python engine *and* against the independently computed Excel figures, so agreement is not an artefact of one implementation being written to satisfy them.

| Identity | Why it must hold | Result |
|---|---|---|
| TCPI against EAC1 = CPI | Substituting EAC = BAC/CPI into TCPI = (BAC−EV)/(EAC−AC) reduces to CPI exactly. A forecast built on "performance continues" demands, by construction, no change in performance. | Holds in both, to 1e−9 |

| Identity | Why it must hold | Result |
|---|---|---|
| VAC on EAC2 = CV | $VAC_2 = BAC - (AC + BAC - EV) = EV - AC = CV$. If the rest runs at budget, the final overrun is exactly the overrun already incurred. | Holds in both, to ₹0.01 |
| TCPI against EAC2 = 1.000 | EAC2 assumes the remainder runs at budget, so it requires efficiency of exactly 1.00. | Holds exactly |

---

## 5. Discrepancies found, and how each was resolved

Four genuine problems surfaced during validation. All are recorded, including the ones that turned out to be our own error rather than the tool's.

### 5.1 EAC method 3 differed by ₹156 — rounding in the calculation chain

**Found.** The hand calculation gave EAC3 = ₹17,98,32,249. The engine gave ₹17,98,32,093. A difference of ₹156.

**Investigated.** Rather than assume either side was right, the quantity was recomputed with 40-digit decimal arithmetic:

```
CPI × SPI, full precision     0.7520247619957480232336676...
CPI × SPI, rounded to 6 dp    0.752025
EAC3 from full precision      179,832,092.77
EAC3 from the rounded index   179,832,048.47
```

**Cause.** The hand calculation had rounded the composite index CPI × SPI to six decimal places mid-chain — which is what one naturally does on paper, and looks harmless. It is not: because EAC3 divides a ₹10.5 crore remainder by that index, a change in the seventh decimal place moves the forecast by tens of rupees. Our cruder rounding moved it by ₹156.

**Resolved.** The engine was correct. The hand figure was corrected. More usefully, a rule was adopted for the rest of the work: **rounding belongs in presentation only, never in the calculation chain.** The benchmark workbook applies rounding through cell number formats alone, and the engine never rounds. A regression test (`test_rounding_an_intermediate_index_moves_the_forecast`) pins the lesson so it cannot quietly return.

**Why it is worth reporting despite being immaterial.** ₹156 on a ₹18 crore forecast is nothing. But the same habit applied to CPI itself early in a project, when EV is small and the divisor is volatile, moves the forecast by lakhs. The discrepancy was trivial; the practice behind it was not.

### 5.2 Two validation tests were mis-constructed, not the engine

**Found.** Two tests of the input validation rules failed, apparently showing the wrong error being raised.

**Investigated.**

- `test_package_budgets_must_sum_to_the_declared_bac` mutated a work package budget. That broke per-package reconciliation, which is checked *earlier*, so that error fired first.

- `test_actuals_may_not_run_past_the_data_date` appended a row copied from Period 1, carrying a percent complete lower than the package had already reached. That tripped the monotonicity rule first.

**Resolved.** In both cases the engine's behaviour was correct and the test was wrong. The tests were rewritten to exercise each rule in isolation — the first by altering the declared BAC rather than a package budget, the second by carrying progress forward rather than reversing it. **No engine code was changed.**

**Why it is reported.** It is the failure mode the brief warns about, in miniature: a test failed, and the tempting move is to "fix" the code until the test passes. Doing so here would have *removed* a correct validation rule to satisfy a badly written test.

### 5.3 Executive summary invisible under a dark browser theme

**Found.** Only by looking at the running application. The custom CSS set light panel backgrounds without setting a text colour, so when the browser requested dark mode Streamlit supplied white text, and the executive summary rendered white-on-white — an empty box where the most important paragraph on the screen should be.

**Resolved.** Explicit `color` on every styled panel, and the theme pinned to light in `.streamlit/config.toml`, with the reasoning recorded there: the figures get screenshotted into progress reports and printed, and a dark dashboard prints as a black rectangle.

**Why it is reported.** No unit test would have caught it. The numbers were all correct; the tool was simply unusable. It is a reminder that "validated" and "works" are different claims.

### 5.4 Progress carry-forward logic was rewritten before it could fail

**Found.** In review rather than by a failing test. The routine filling gaps in reported progress had grown a nested loop with a conditional that indexed a pivot table by a column that might not exist.

**Resolved.** Replaced with a forward-fill along the period axis followed by a zero-fill. The semantics are now stated plainly in the code: a missing period means progress is unchanged, and anything before the first report is zero.

**Why it is reported.** It had not produced a wrong answer, because the case study has no gaps in reporting. Real project data does. The defect would have surfaced in exactly the situation the tool is meant to handle.

### 5.5 A project under budget was told to improve by –5%

**Found.** By building a second case study specifically to exercise the branches the first one never reaches. The bundled project is adverse on both cost and schedule, so every favourable path through the commentary was unexercised. A second synthetic project — a 15 MLD water treatment plant, behind programme but **under** budget at CPI 1.041 — produced this closing sentence in its executive summary:

> Returning to budget would require a **−5% improvement** in cost efficiency on all remaining work.

**Cause.** The sentence was generated unconditionally from TCPI divided by CPI. When a project is already forecast to underrun, that ratio is below one and the "improvement required" is negative, which is not a quantity that exists.

**Resolved.** The closing sentence now branches. A project forecast to overrun is told what improvement is required; a project forecast to underrun is told how much headroom it has — "*the remaining work could run about 5% less cost efficiently than it has so far and still finish within budget*". A regression test (`tests/test_commentary.py::test_under_budget_project_is_not_asked_to_improve`) fixes the behaviour, and the second case study is committed under `samples/case-02-water-treatment/` so the branch stays exercised.

**Why it is reported.** Every number in the sentence was correct. The arithmetic was right and the English was nonsense, and no comparison against the benchmark could have caught it, because the benchmark checks figures and not prose. It is the same class of defect as 5.3: output that is numerically valid and unusable by a reader. It also makes a narrower point about our own test data — a single case study, however carefully built, only tests the paths it happens to take.

### 5.6 The Excel export appeared to download an empty file

**Found.** By a reader opening the exported workbook, which is the only way this could have been found. Every sheet showed its labels and not one number.

**Cause.** `openpyxl` stores a cell as *either* a formula *or* a value, never both, so every formula we write is saved with no cached result. Real Excel stores both: the formula and the last computed value. We set `fullCalcOnLoad`, which makes Excel and LibreOffice recalculate when the file opens, and there the workbook displays correctly. Anything that reads cached values without calculating — Apple Numbers, macOS Quick Look, most in-browser previewers — finds nothing to read. Inspecting the file confirmed it: 46 populated cells on the Dashboard sheet, of which 29 had cached values, and all 29 were text labels. Passing the same file through LibreOffice headless raised that to 46.

**Resolved.** The live-formula behaviour is deliberate and was kept — it is the reason the workbook can be interrogated without the tool. What was missing was any way for the reader to know. The Dashboard sheet now carries a note at the top, the Read Me opens with an "*If the dashboard looks empty*" section, and the headline figures are written to the Read Me as plain numbers so that something is always readable. The dashboard's download button repeats the warning.

**Why it is reported.** The file was valid the entire time. Every formula was correct, and the workbook had been checked by reading it back with the same library that wrote it — which recalculates nothing and so could never have caught this. It took someone opening the file in an ordinary viewer. That is the same lesson as 5.3: we had tested the artefact we produced, not the artefact as it is received.

**Found at the same time.** Two sidebar controls looked inert. Both in fact worked — the breakdown radio changes the variance table from 6 summary rows to 19 packages, and the SPI(t) checkbox moves EAC3 from ₹17.98 crore to ₹17.51 crore — but each affects exactly one tab, so changing one while reading any other tab does nothing visible. The checkbox was also mislabelled: it reads "EAC method 3" and silently changes EAC4 as well, because EAC4 blends the same index. Each control now names the tab it governs, and the checkbox names both methods.

---

## 6. What this validation does not establish

This section matters more than section 4.

**It validates the calculation. It does not validate the inputs.**

The Python engine and the Excel benchmark read the same budgets, the same percent-complete figures and the same costs. That is deliberate — a benchmark against different data would compare nothing. What has been established is that two separately written implementations of the EVM definitions agree. An arithmetic or logic error in one would not be reproduced by the other, and none was found.

But if a work package is reported at 60% complete when it is really 40%:

- the Python engine computes a wrong EV,
- the Excel benchmark computes **the same** wrong EV,
- all 50 comparisons agree perfectly,
- and every index on the dashboard is confidently, precisely wrong.

**No spreadsheet and no test suite can detect this. Only a measurement at site can.**

The practical consequence for anyone deploying such a tool is that the governance around how percent complete is assessed, by whom, and with what independent check, matters more than the software — and it is the part most likely to be neglected, because it is the part that cannot be bought.

Three further limits of this exercise:

1. **One case study.** The benchmark exercises one project at two data dates. It does not cover re-baselining, gaps in reporting, packages whose scope changed, or negative corrections.
2. **Chart appearance is not machine-checkable.** The tests assert that each Excel chart default known to produce a broken chart has been overridden. Nothing here renders Excel, so whether the chart is *legible* still needs a human to open the file. We opened it.
3. **The data is synthetic.** The tool has not been tested against the messiness of a real cost ledger.

---

## 7. Reproducing this

```
.venv/bin/python data/generate_case_study.py        # regenerate inputs
.venv/bin/python benchmark/build_benchmark.py       # place the formulas
.venv/bin/python scripts/validate_benchmark.py      # evaluate and compare
.venv/bin/python -m pytest tests/ -v                # 151 tests
```

The third command needs LibreOffice. The test suite does not — it reads a committed snapshot of the LibreOffice-computed values, with a test asserting the snapshot is not older than the workbook it describes.